

Software Engineering

How To Create Software Products Everyone Loves!

Module 18

Software Architecture and Technical Design: OOP Design Patterns

Ali Samanipour

May. 2023

Ali Samanipour
[linkedin.com/in/Samanipour](https://www.linkedin.com/in/Samanipour)

1

SOLID Principles

2

Creational Design Patterns: Factory Method

3

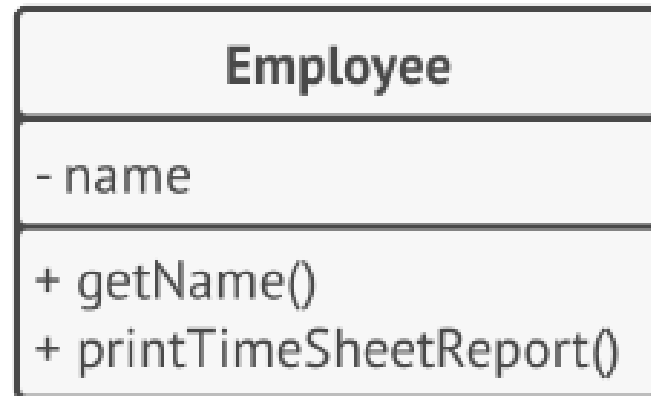
Structural Design Patterns: Adapter

4

Behavioral Design Patterns: Chain of Responsibility

Single Responsibility Principle

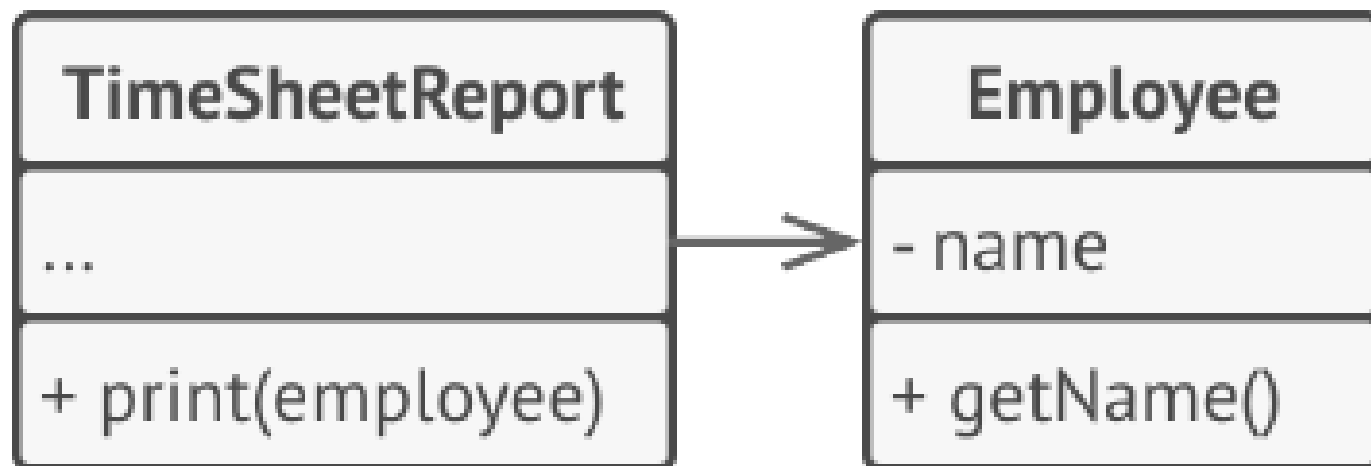
A class should have just one reason to change.



BEFORE: the class contains several different behaviors.

Single Responsibility Principle

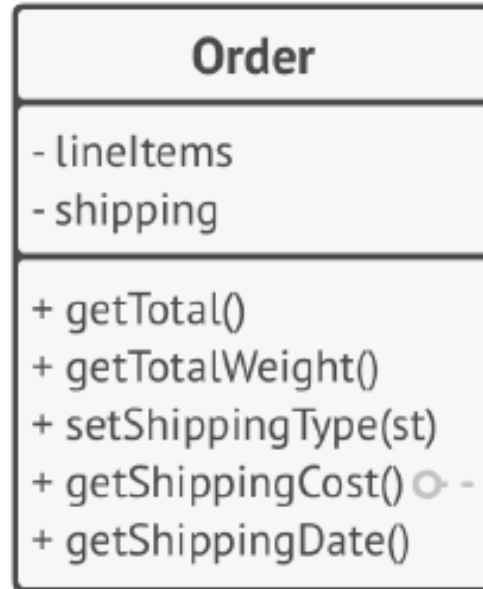
A class should have just one reason to change.



AFTER: the extra behavior is in its own class.

Open/Closed Principle

Classes should be open for extension but closed for modification.



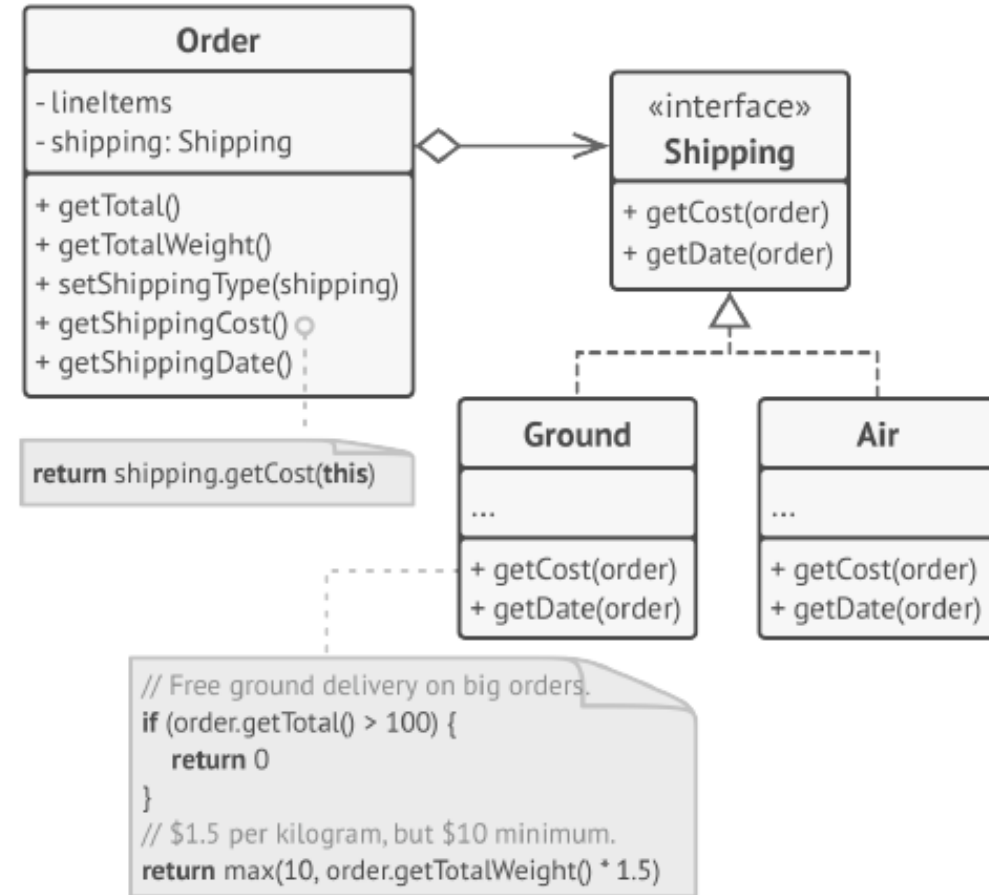
```
if (shipping == "ground") {
    // Free ground delivery on big orders.
    if (getTotal() > 100) {
        return 0
    }
    // $1.5 per kilogram, but $10 minimum.
    return max(10, getTotalWeight() * 1.5)
}

if (shipping == "air") {
    // $3 per kilogram, but $20 minimum.
    return max(20, getTotalWeight() * 3)
}
```

BEFORE: you have to change the `Order` class whenever you add a new shipping method to the app.

Open/Closed Principle

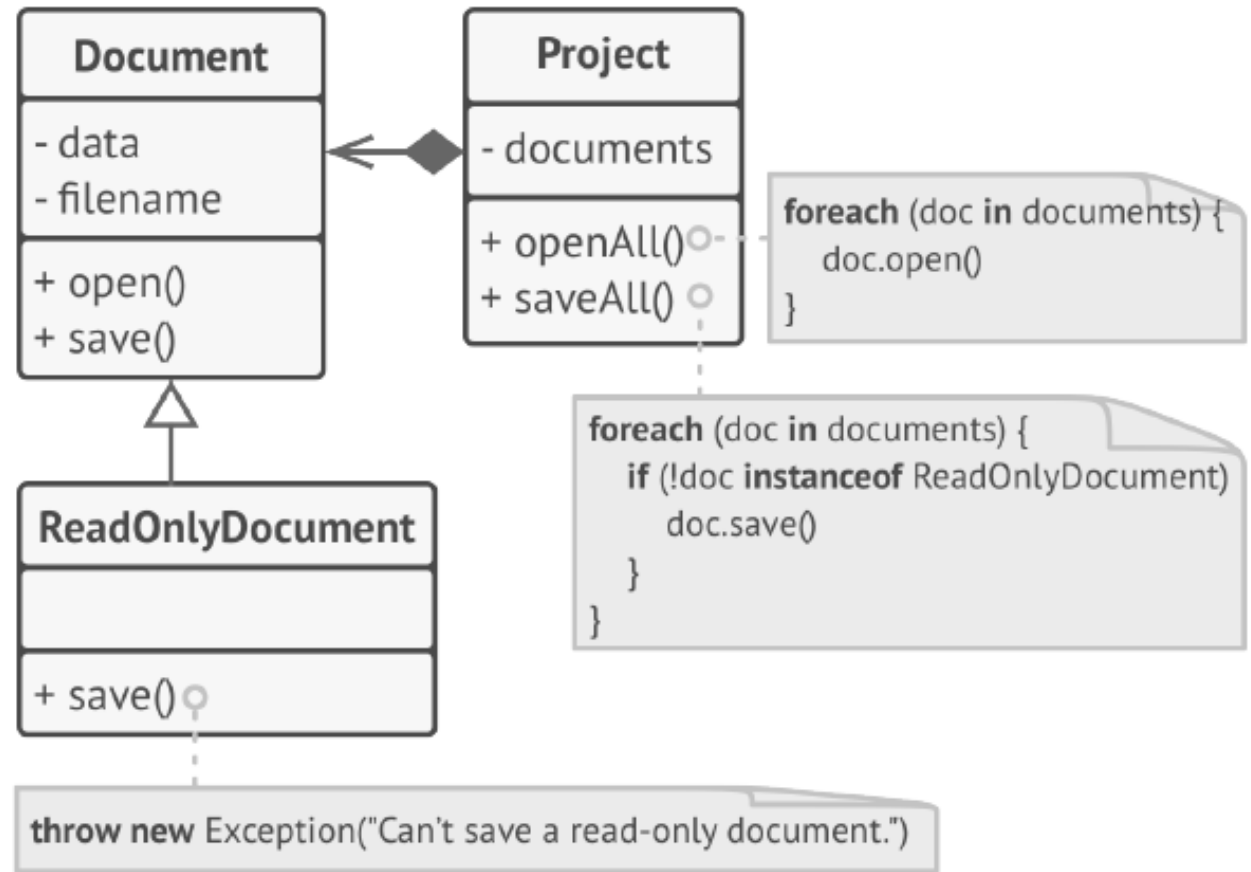
Classes should be open for extension but closed for modification.



AFTER: adding a new shipping method doesn't require changing existing classes.

Liskov Substitution Principle

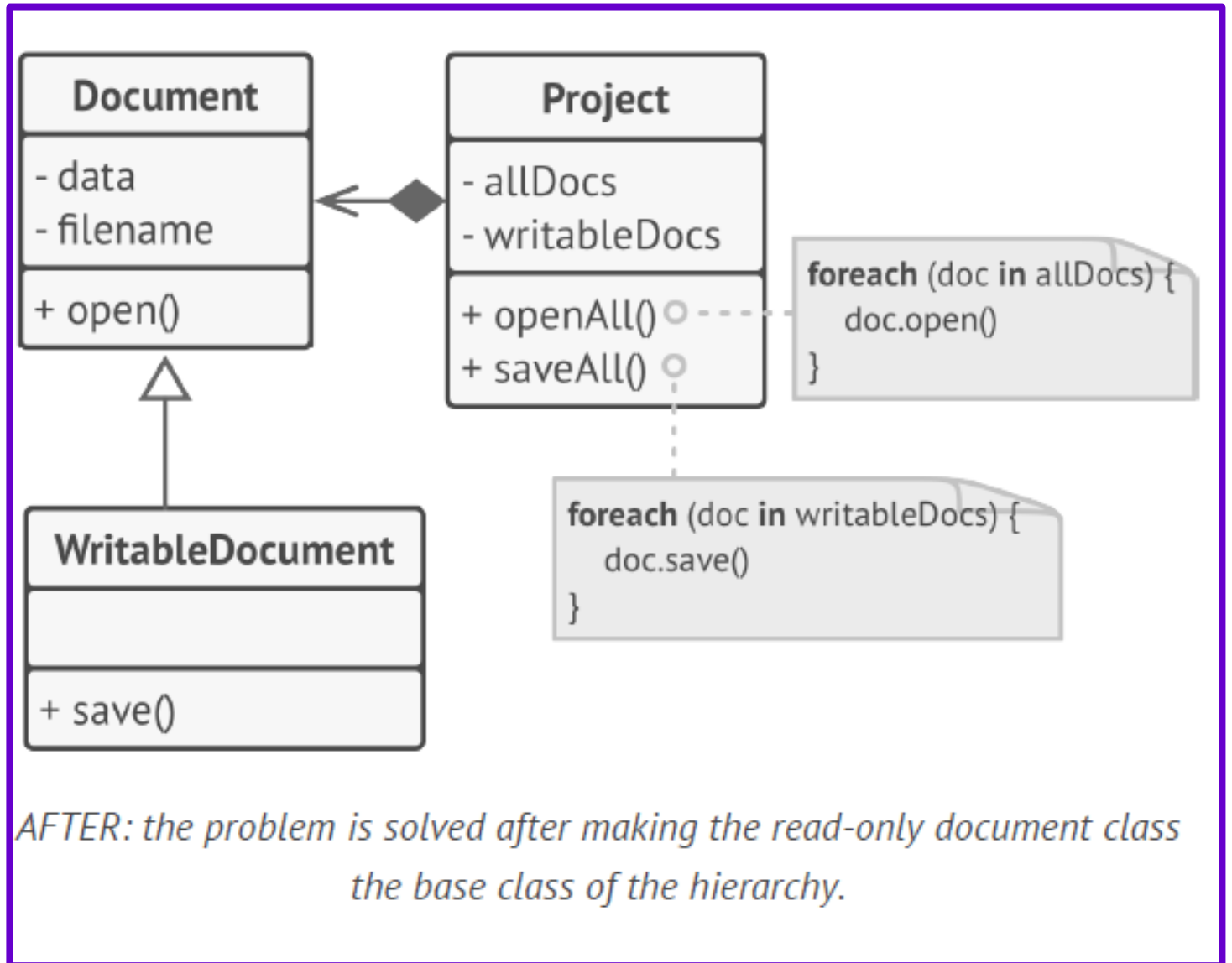
When extending a class, remember that you should be able to pass objects of the subclass in place of objects of the parent class without breaking the client code.



BEFORE: saving doesn't make sense in a read-only document, so the subclass tries to solve it by resetting the base behavior in the overridden method.

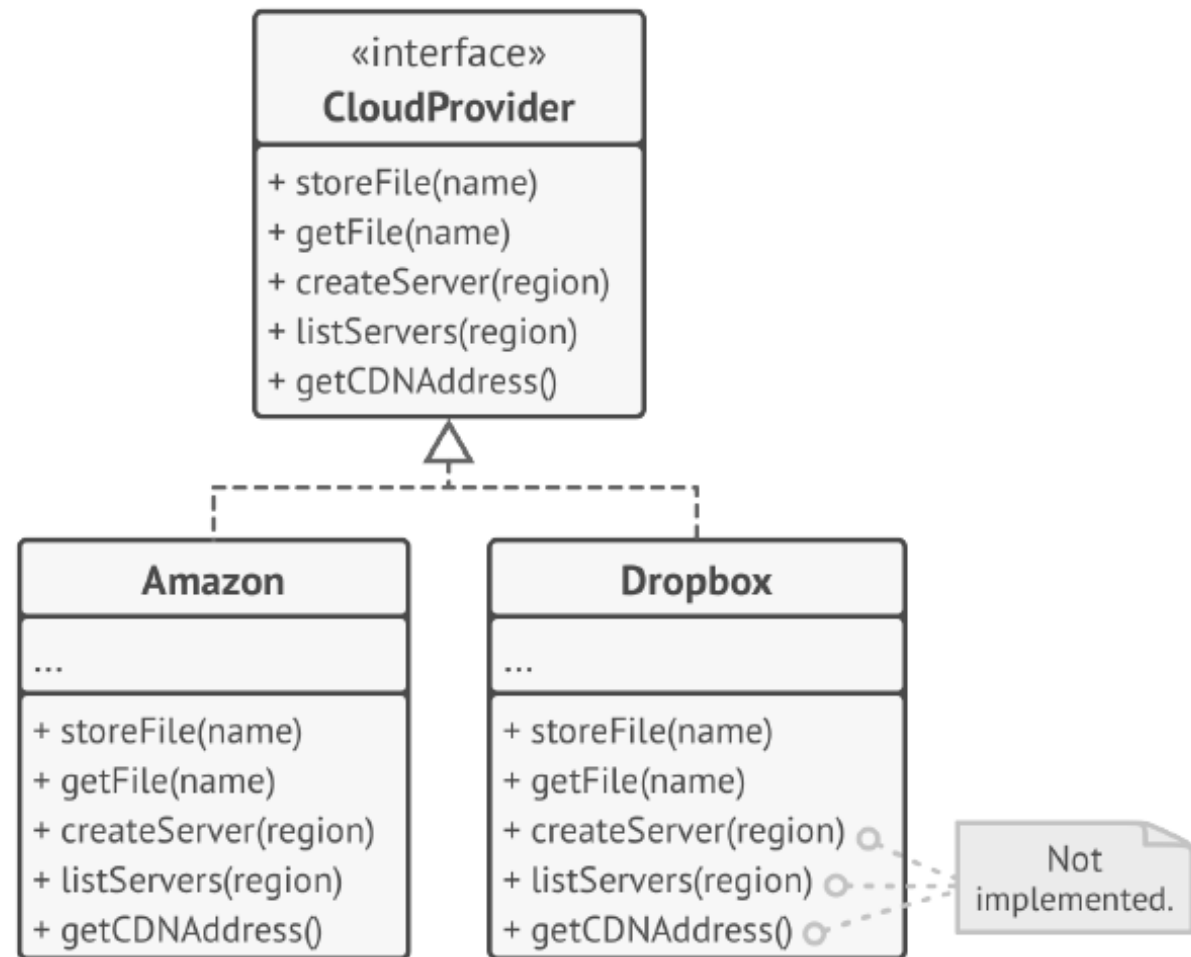
Liskov Substitution Principle

When extending a class, remember that you should be able to pass objects of the subclass in place of objects of the parent class without breaking the client code.



Interface Segregation

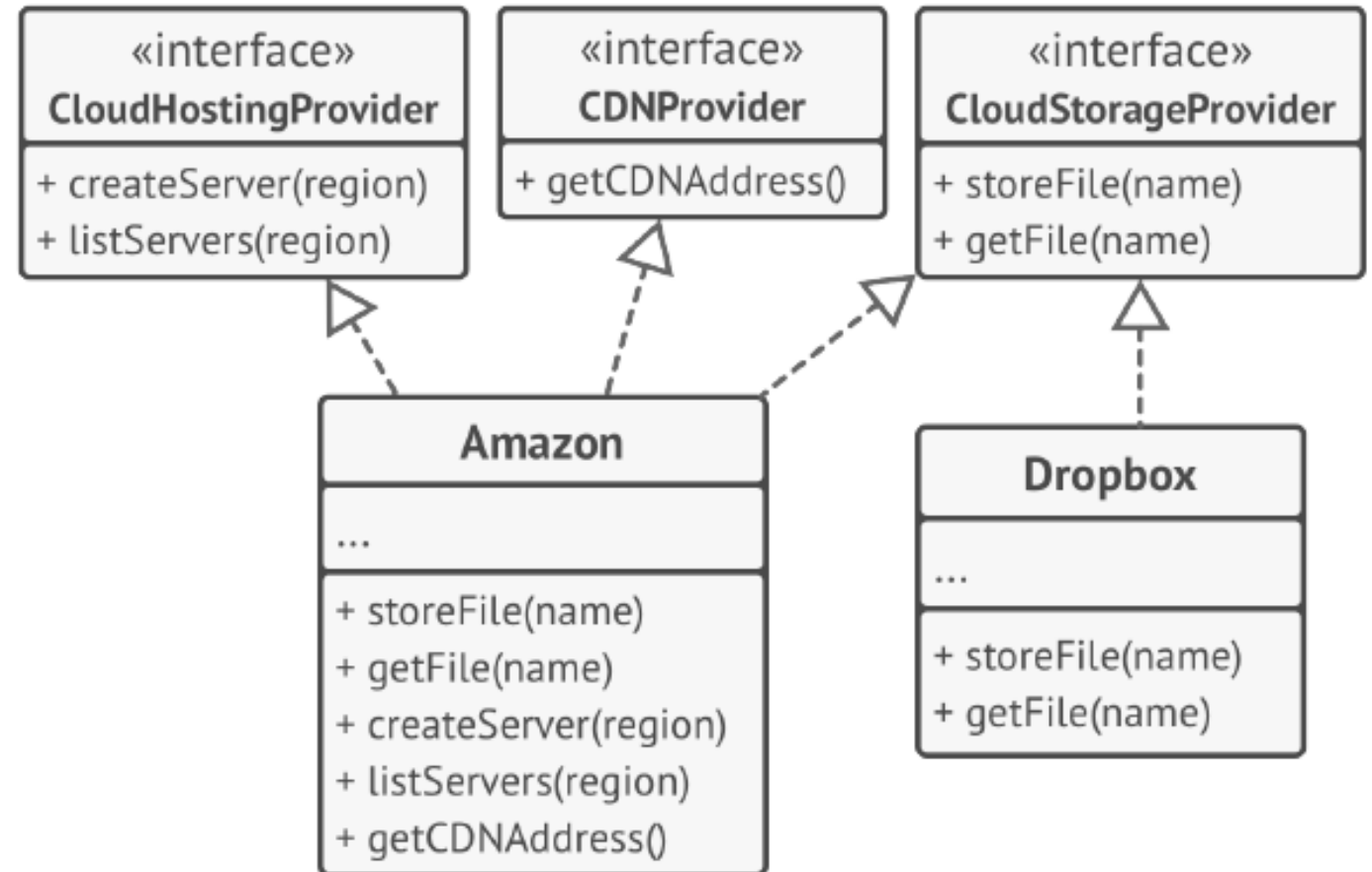
Clients shouldn't be forced to depend on methods they do not use.



BEFORE: not all clients can satisfy the requirements of the bloated interface.

Interface Segregation

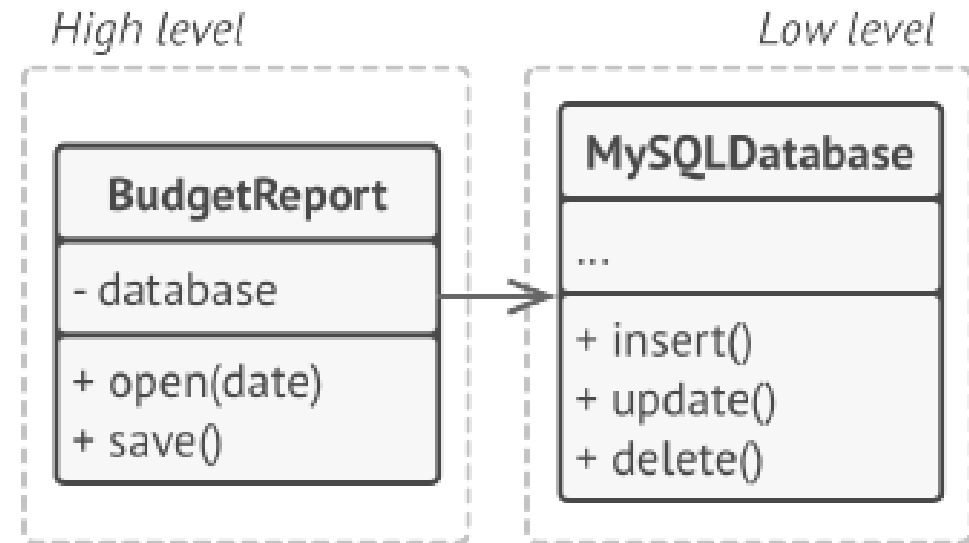
Clients shouldn't be forced to depend on methods they do not use.



AFTER: one bloated interface is broken down into a set of more granular interfaces.

Dependency Inversion Principle

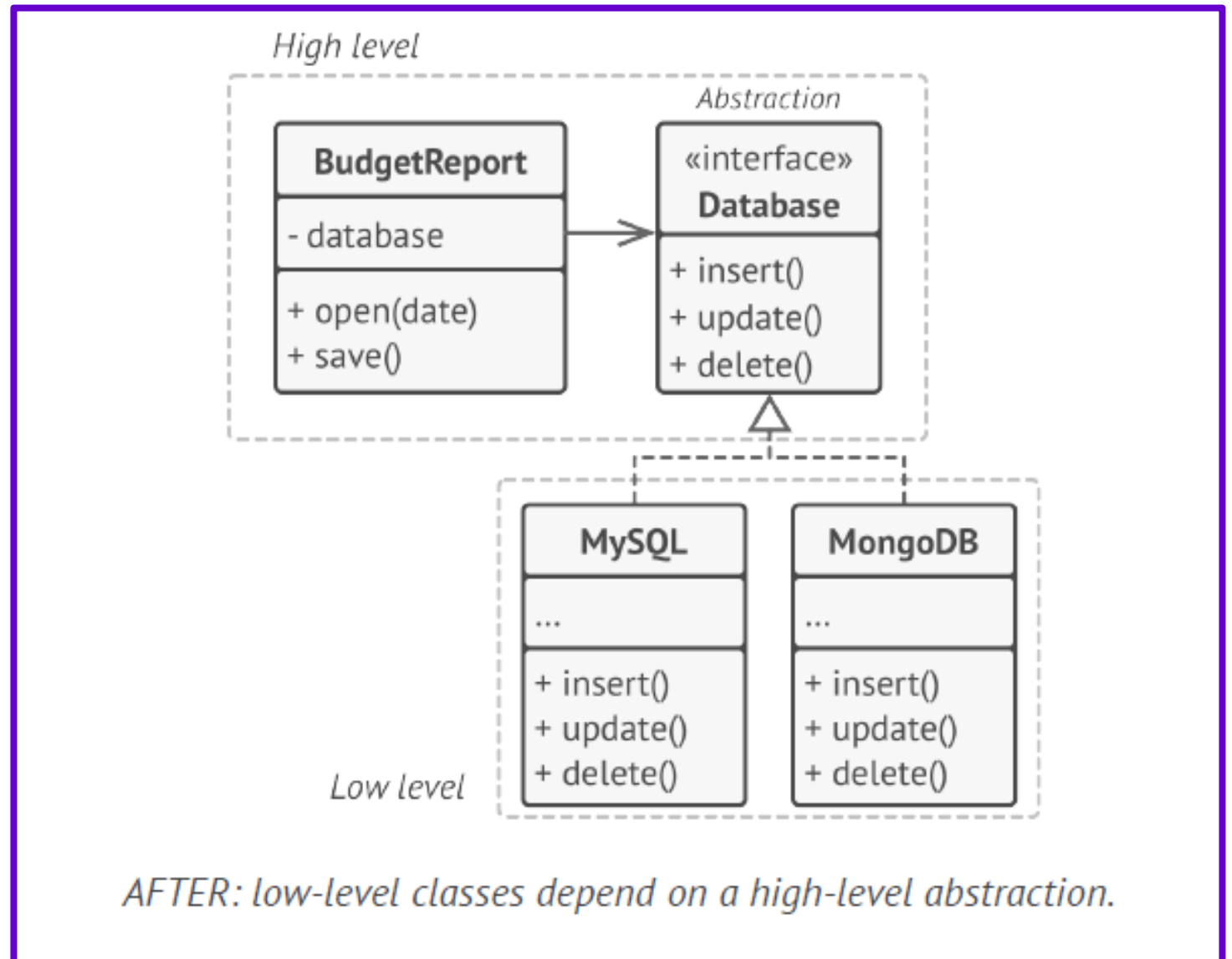
High-level classes shouldn't depend on low-level classes. Both should depend on abstractions. Abstractions shouldn't depend on details. Details should depend on abstractions.



BEFORE: a high-level class depends on a low-level class.

Dependency Inversion Principle

High-level classes shouldn't depend on low-level classes. Both should depend on abstractions. Abstractions shouldn't depend on details. Details should depend on abstractions.



1

SOLID Principles

2

Creational Design Patterns: Factory Method

3

Structural Design Patterns: Adapter

4

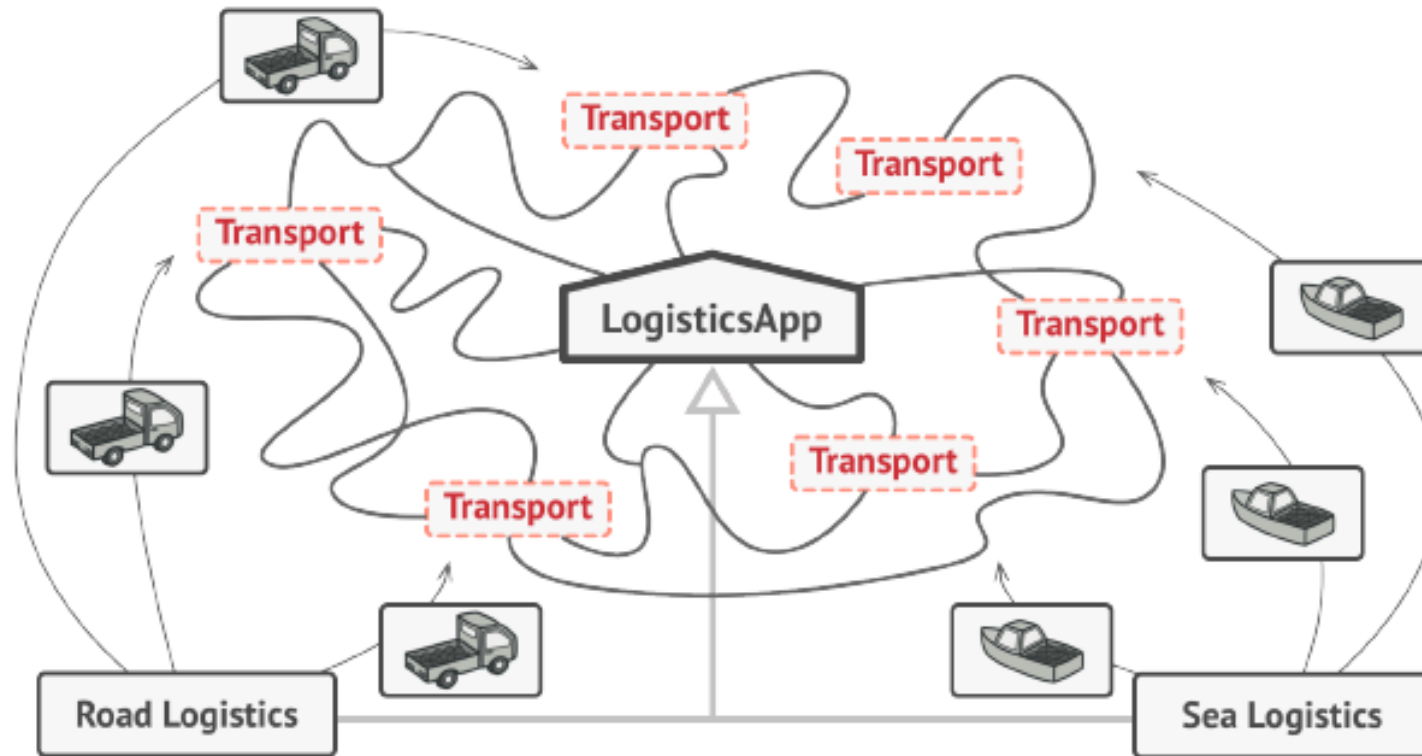
Behavioral Design Patterns: Chain of Responsibility

Factory Method Pattern: Problem



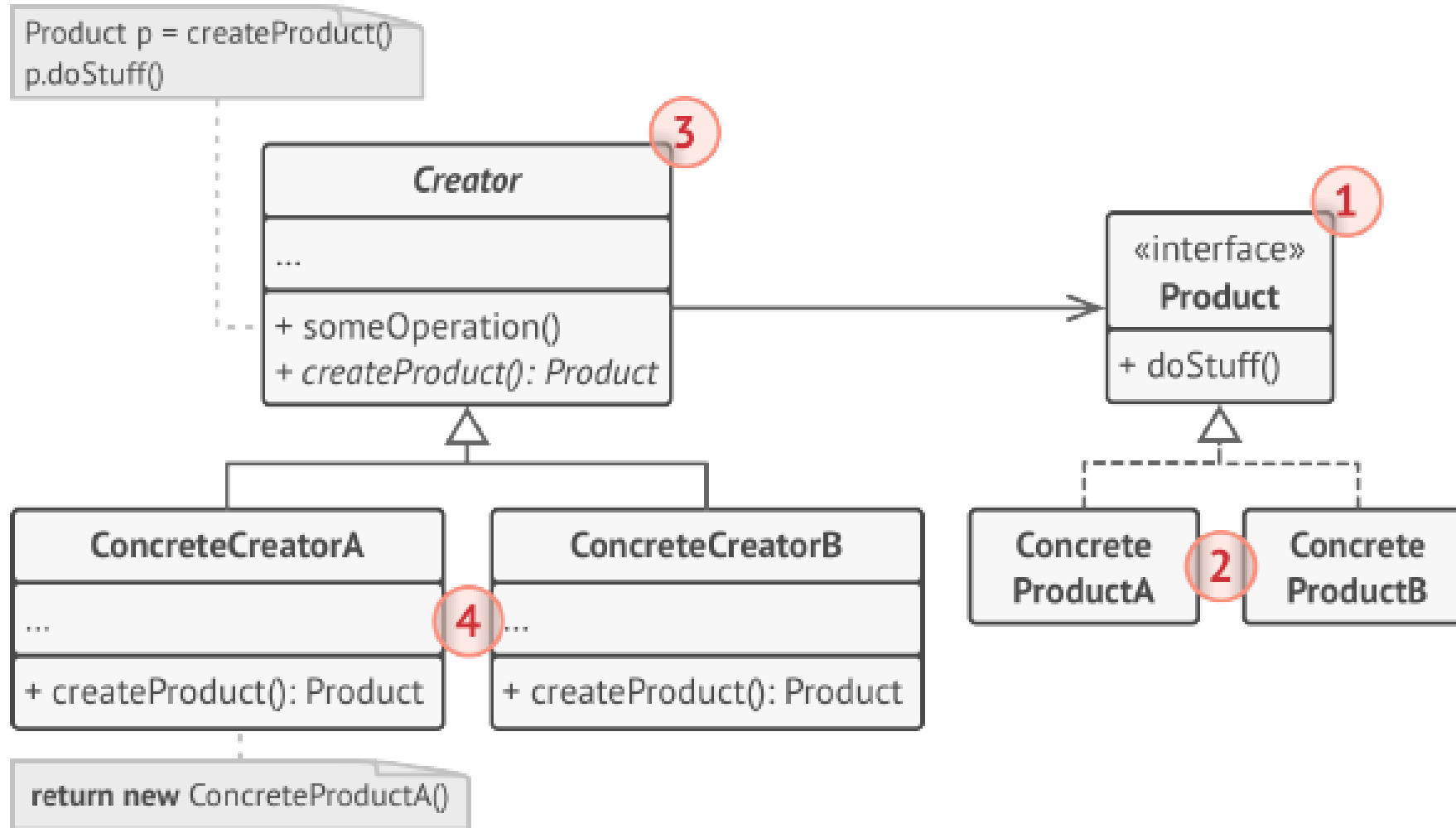
Adding a new class to the program isn't that simple if the rest of the code is already coupled to existing classes.

Factory Method Pattern: Solution

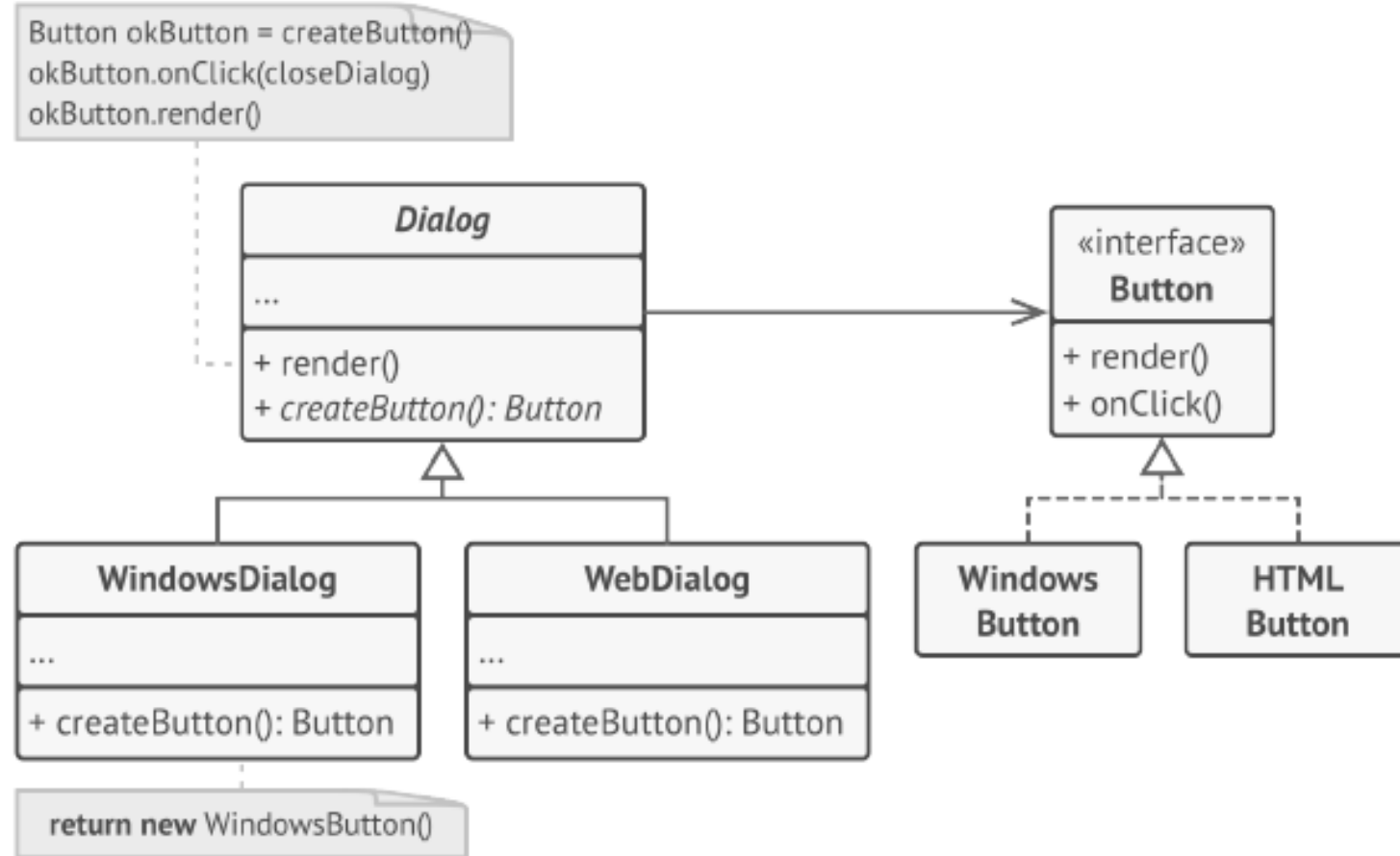


As long as all product classes implement a common interface, you can pass their objects to the client code without breaking it.

Factory Method Pattern: Structure



Factory Method Pattern: Example 2



The cross-platform dialog example.

1

SOLID Principles

2

Creational Design Patterns: Factory Method

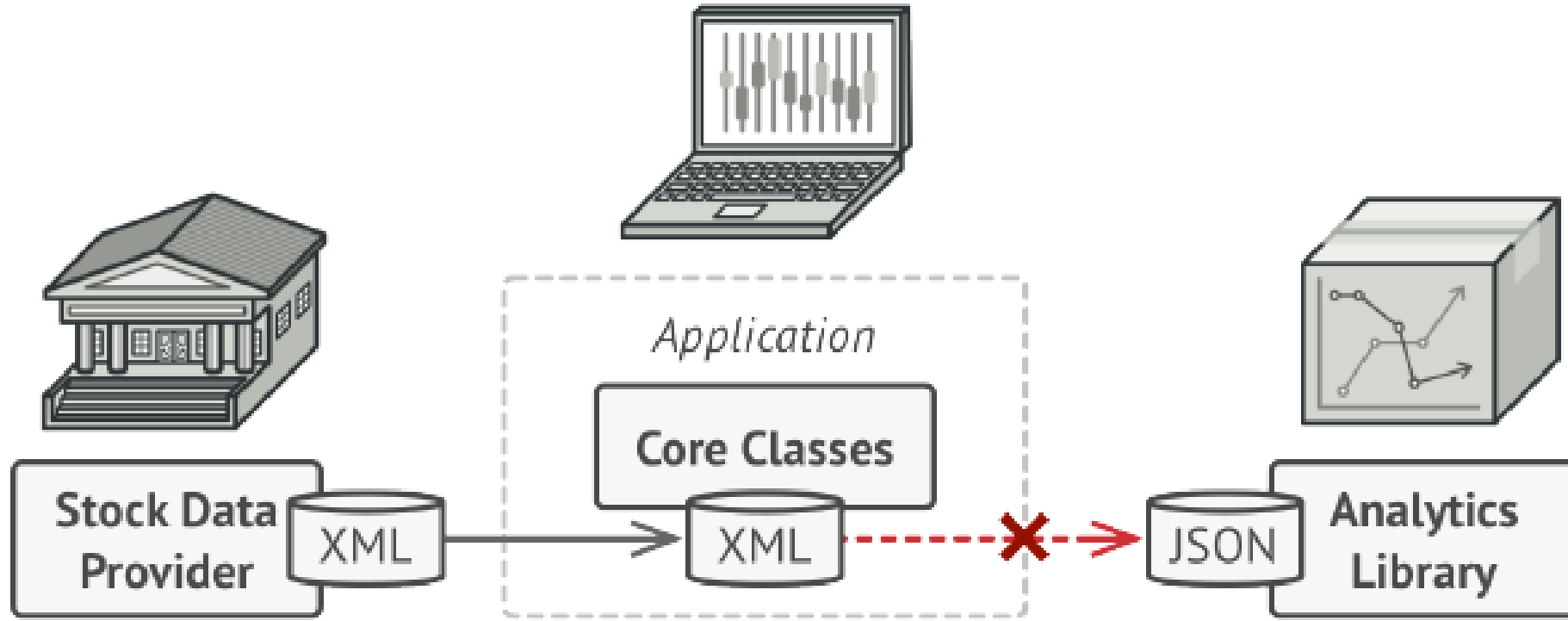
3

Structural Design Patterns: Adapter

4

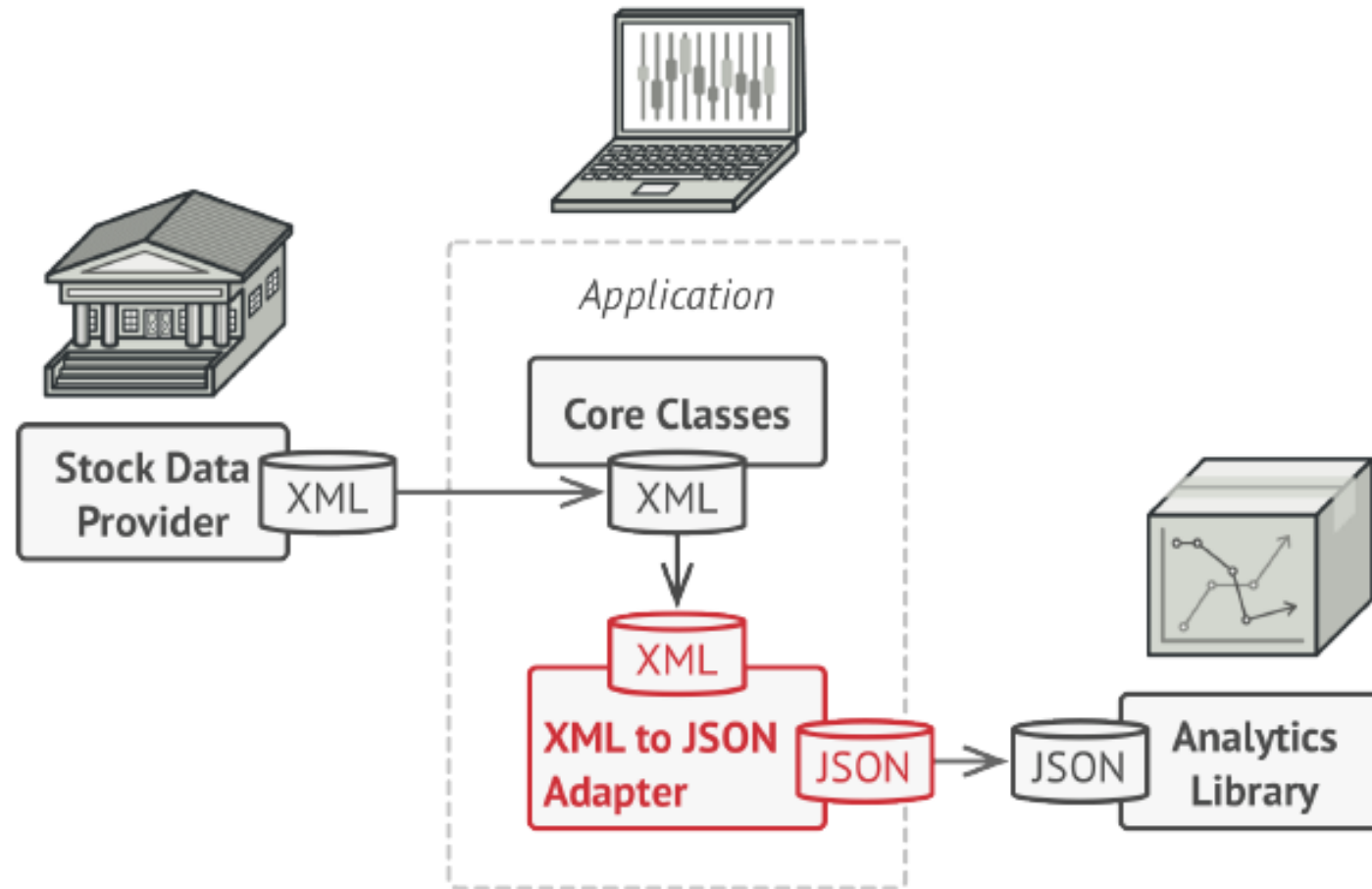
Behavioral Design Patterns: Chain of Responsibility

Adapter Method Pattern: Problem

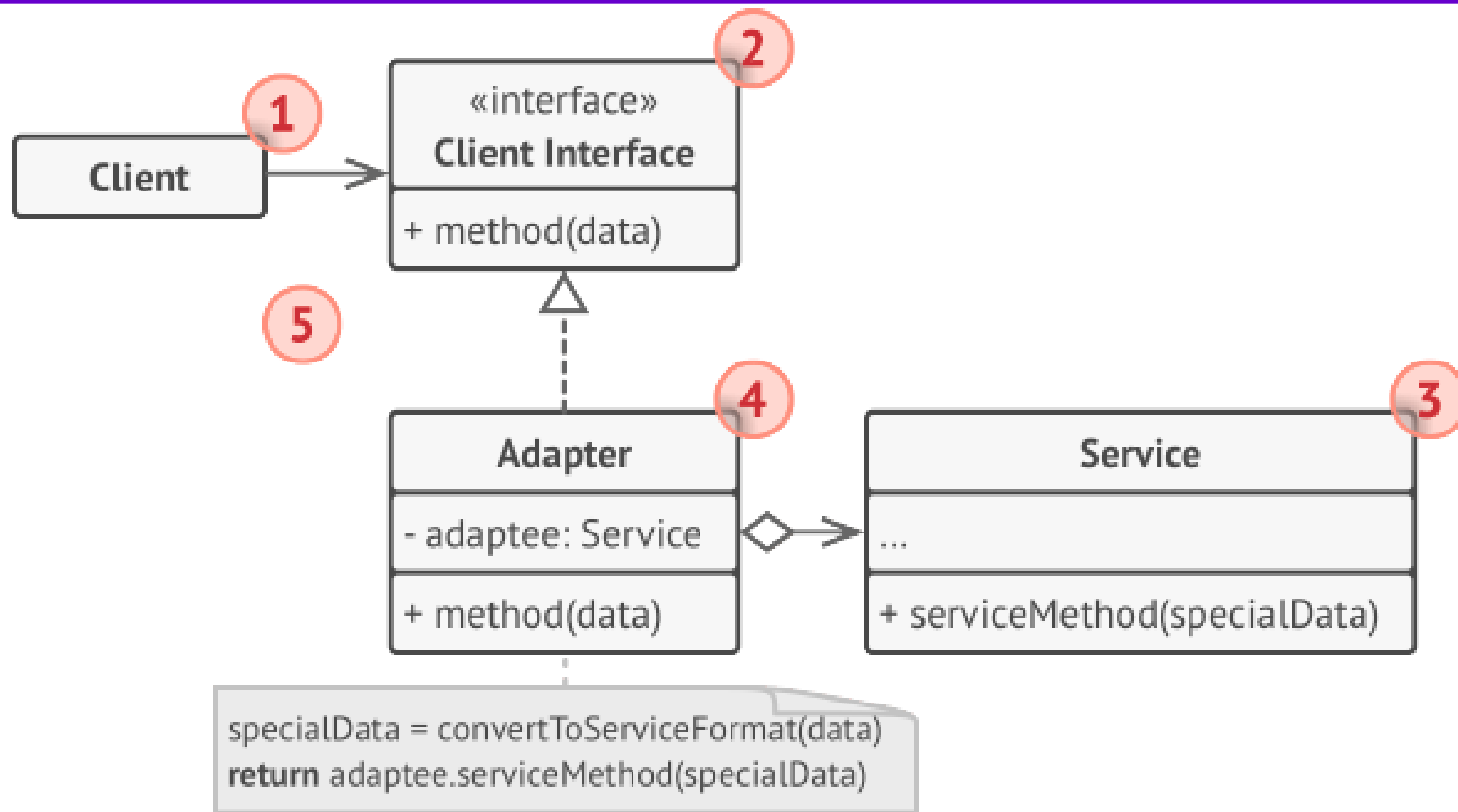


You can't use the analytics library "as is" because it expects the data in a format that's incompatible with your app.

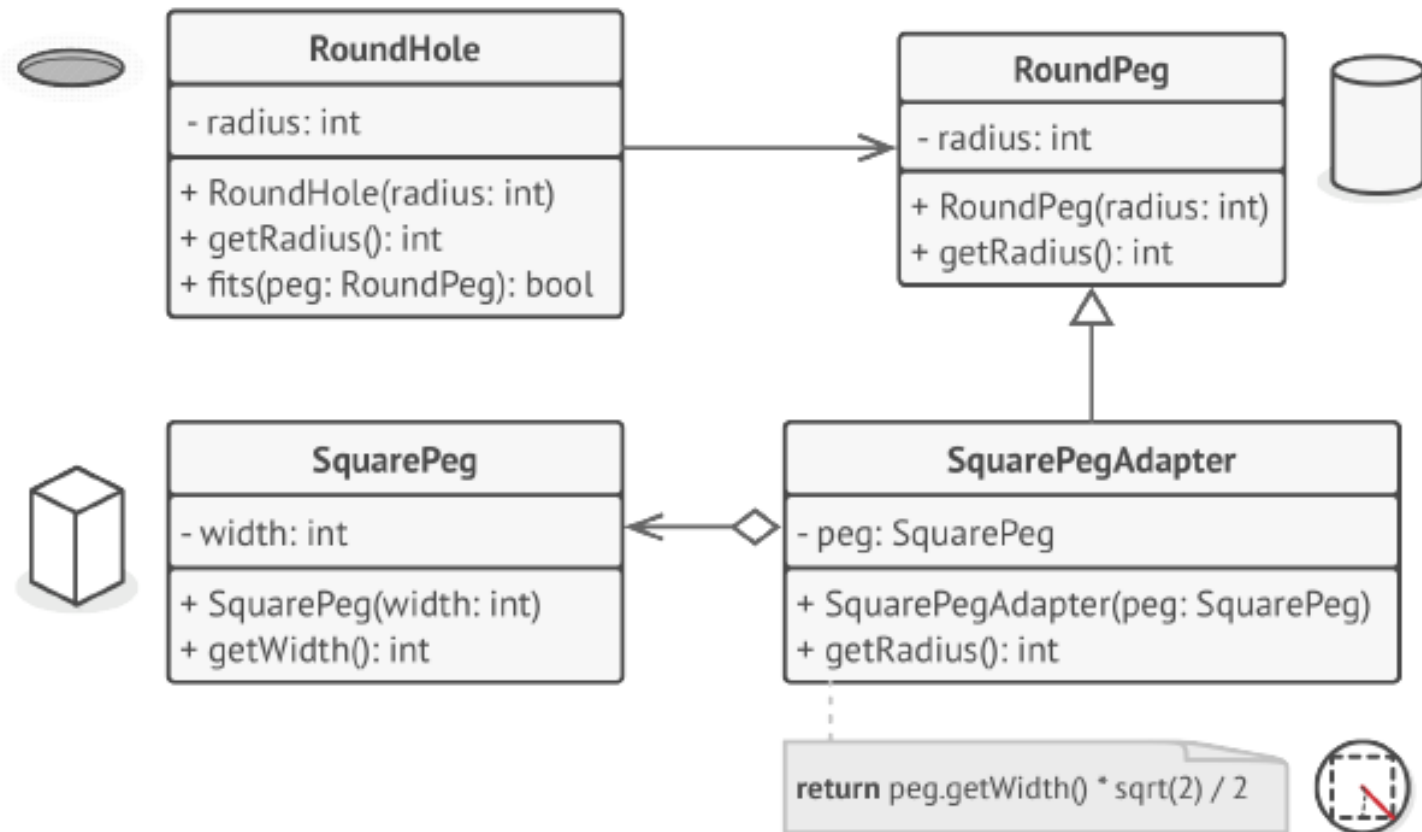
Adapter Method Pattern: Solution



Adapter Method Pattern: Structure



Adapter Method Pattern: Example



Adapting square pegs to round holes.

1

SOLID Principles

2

Creational Design Patterns: Factory Method

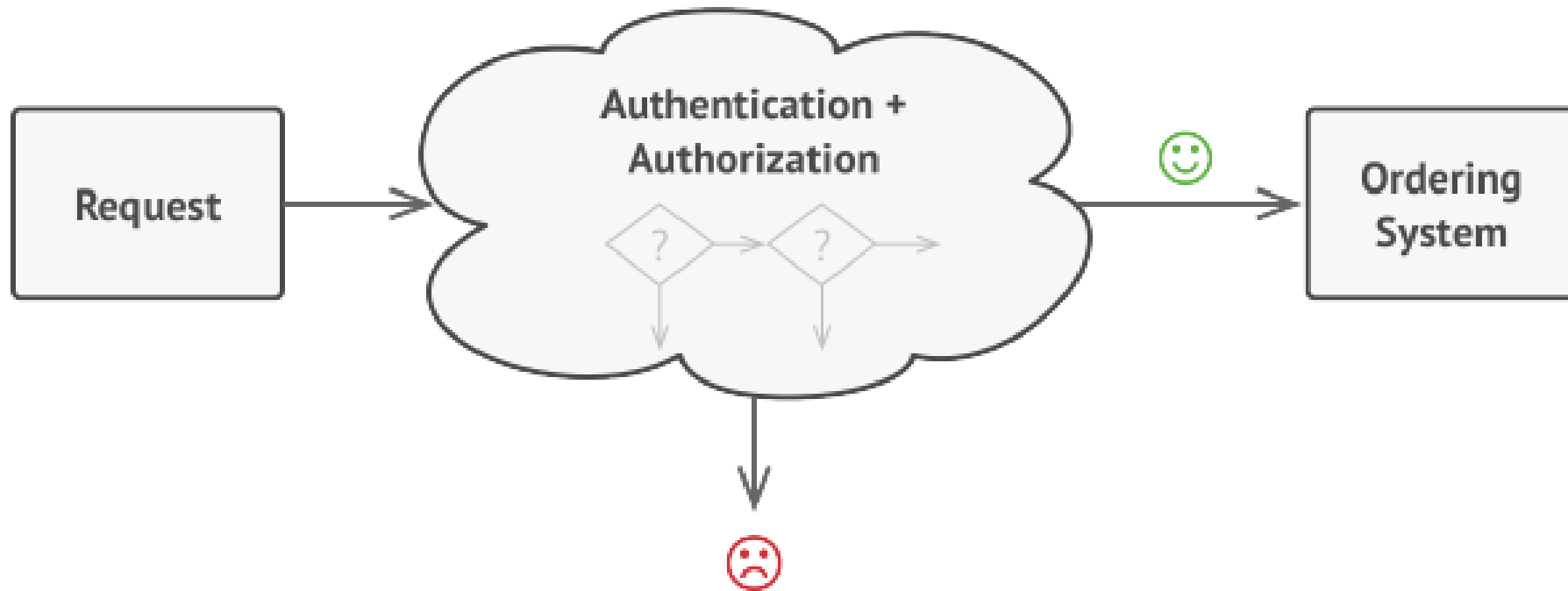
3

Structural Design Patterns: Adapter

4

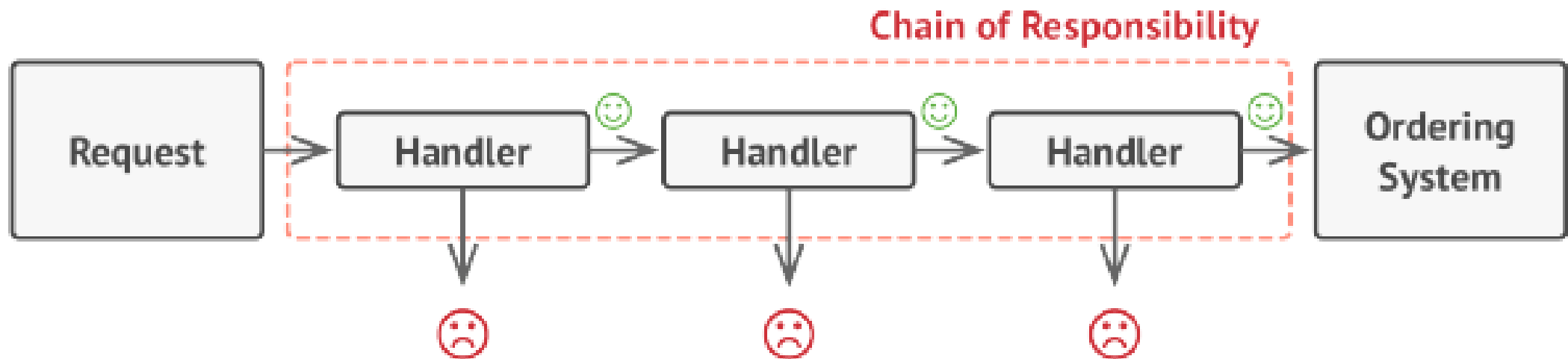
Behavioral Design Patterns: Chain of Responsibility

Chain of Responsibility Pattern: Problem



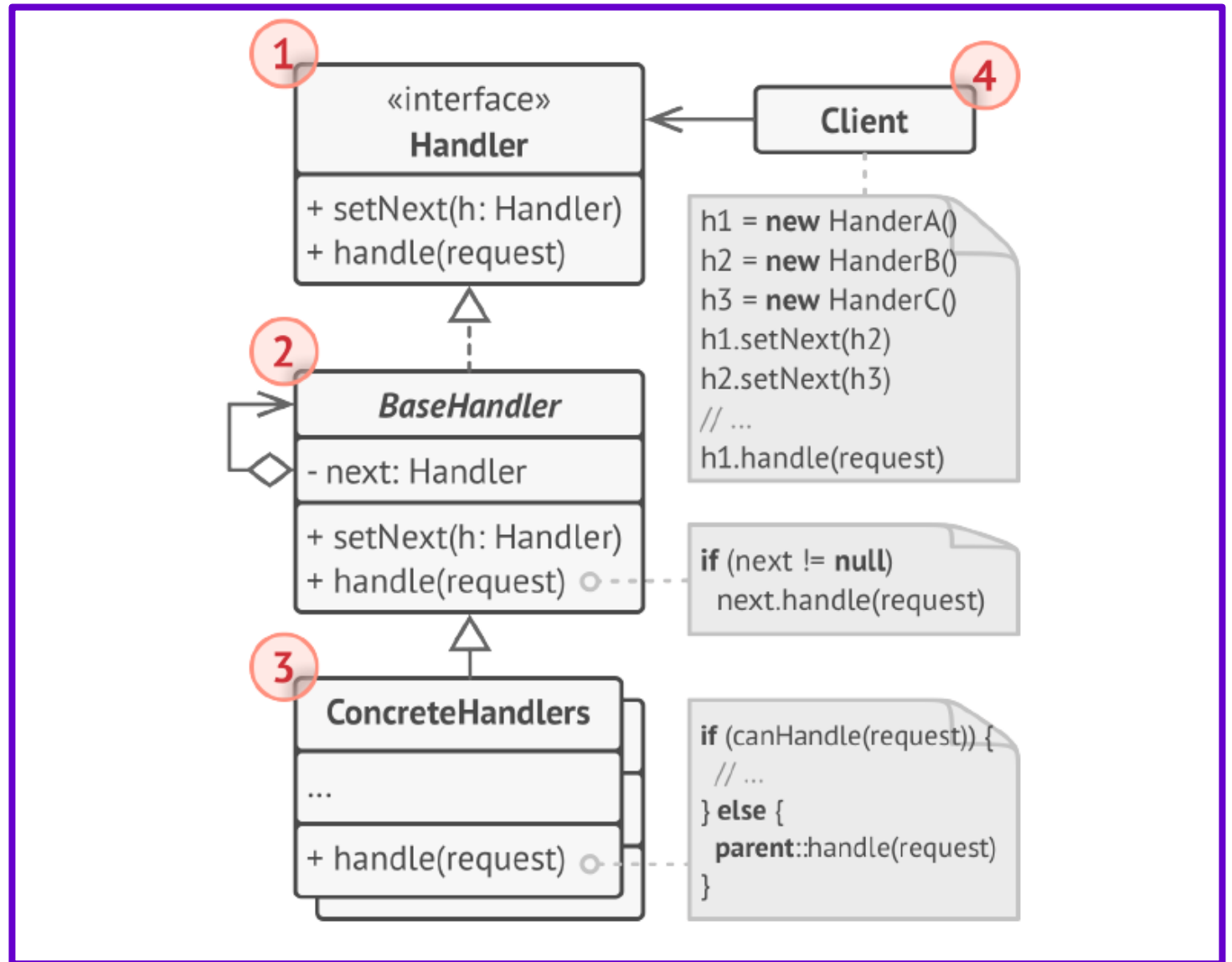
The request must pass a series of checks before the ordering system itself can handle it.

Chain of Responsibility Pattern: Solution

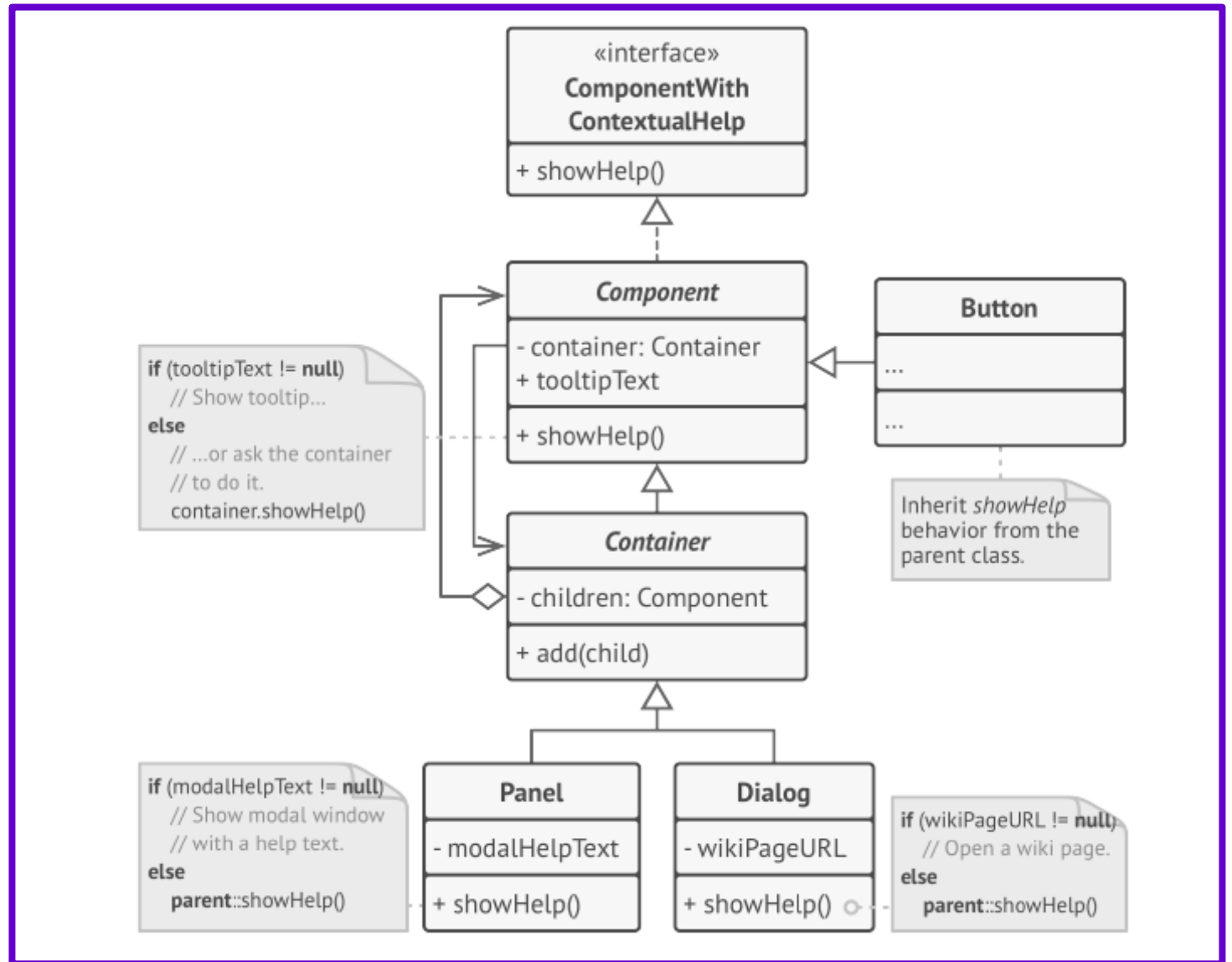


Handlers are lined up one by one, forming a chain.

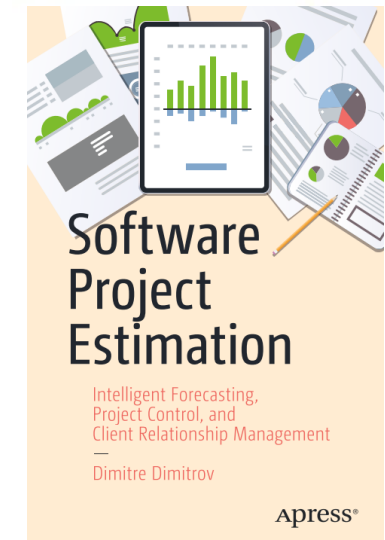
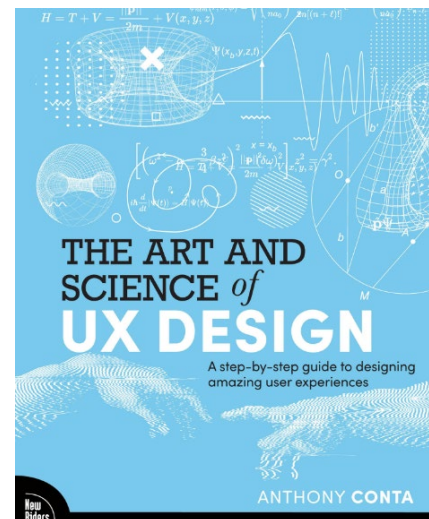
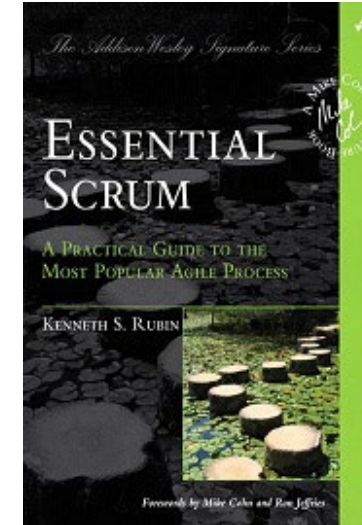
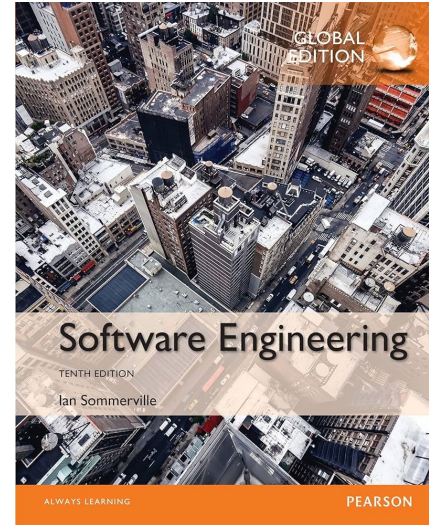
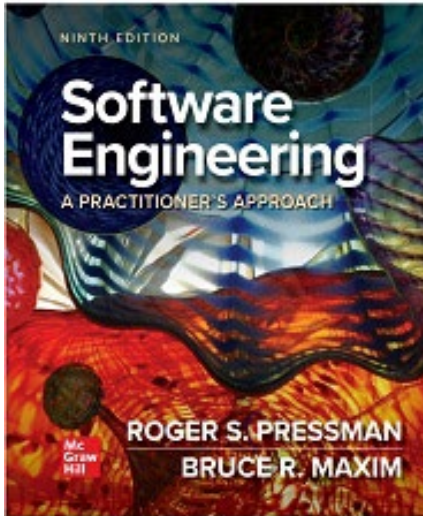
Chain of Responsibility Pattern: Structure



Chain of Responsibility Pattern: Example



Course References



Course References

- ***Designing User Interfaces***, Michal Malewicz & Diana Malewice, 2020
- ***UI Design Styles: Trends and Design Patterns***, Michal Malewicz & Diana Malewice, 2020
- ***What UX Is Really About :Introducing a Mindset for Great Experiences***, Celia Hodent, CRC Press, 2022
- ***Lean UX: Designing Great Products with Agile Teams 3rd Edition***, Jeff Gothelf & Josh Seiden, O'Reilly, 2021
- ***Laws of UX: Using Psychology to Design Better Products & Services***, Jon Yablonski, O'Reilly, 2020
- ***Designing and Prototyping Interfaces with Figma***, Fabio Staiano, Packet Publishing, 2022

Course References

- [1] Green, M. D. (2016). Scrum: Novice to Ninja: Methods for Agile, Powerful Development, SitePoint.
- [2] Ockerman, S. and S. Reindl (2019). Mastering professional scrum: A practitioner's guide to overcoming challenges and maximizing the benefits of agility, Addison-Wesley Professional.
- [3] Martin, R. C. (2019). Clean Agile, Pearson Education.
- [4] Hall, G. M. (2017). Adaptive Code: Agile coding with design patterns and SOLID principles, Microsoft
- [5] سامانی‌پور، علی. (۲۰۱۸). آموزش اسکرام برای مدیریت چابک فرایند توسعه اپلیکیشن های وب و موبایل،
فرادرس